
Module 11 — Messaging with Kafka

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Topics, Partitions and Offsets	2
2.2 Consumer Groups	2
2.3 Delivery Guarantees	3
3. Step-by-Step Examples	3
3.1 Producing Events	4
3.2 Consuming Events	5
3.3 Errors, Retries and the Dead Letter Topic	6
3.4 Non-Blocking Retries with <code>@RetryableTopic</code>	7
3.5 Kafka Transactions	8
3.6 The Transactional Outbox	9
3.7 Kafka Streams	11
3.8 Testing with Testcontainers	12
4. Common Mistakes and Best Practices	13
5. Summary	13
6. Further Reading	14

1. Learning Goals

By the end of this module you will be able to:

- Explain topics, partitions, offsets and consumer groups
- Send JSON events with `KafkaTemplate` and receive them with `@KafkaListener`
- Handle failures: retries, a dead letter topic (DLT) and poison pills
- Retry without blocking a partition with `@RetryableTopic`
- Write several messages atomically with a Kafka transaction
- Publish events reliably from a database transaction with the transactional outbox pattern
- Build a small Kafka Streams topology and test it without a broker
- Test Kafka code with `Testcontainers`

Prerequisites: Module 06 (PostgreSQL, transactions) · **Estimated time:** 6 hours · **Docker required**

2. Concepts

2.1 Topics, Partitions and Offsets

Kafka is a distributed, append-only **log**. Producers append **records** (key, value, headers) to a **topic**. A topic is split into **partitions**. Within one partition the order is guaranteed, and every record has a growing number, its **offset**.

```
topic "orders"  partition 0:  [0] [1] [2] [3] ...
                  partition 1:  [0] [1] ...
                  partition 2:  [0] [1] [2] ...
```

- The **key** decides the partition: records with the same key always land in the same partition, in order. We use the order id as the key.
- Records are **not deleted** when they are read. They stay until the retention time runs out, so a new consumer can read the past.

2.2 Consumer Groups

A **consumer group** is a set of consumers that share the work of a topic. Every partition is read by exactly one consumer of the group, so a topic with 3 partitions can be processed by at most 3 consumers of a group in parallel. Each group remembers its own offsets.

	Group “shipping”	Group “invoicing”
Receives	every event of orders	every event of orders too
Parallelism	up to 3 instances (3 partitions)	up to 3 instances

Different groups are different applications. Instances of the same application share one group.

2.3 Delivery Guarantees

By default a consumer processes a record and then commits its offset. If it crashes in between, the record is delivered again: **at-least-once**. Consumers must therefore be **idempotent**: processing the same event twice must have the same effect as processing it once (exercise 1).

3. Step-by-Step Examples

With Docker running, start the application. Kafka (KRaft mode, no ZooKeeper) and PostgreSQL start from the root `compose.yaml`:

```
./mvnw -pl modules/11-messaging-kafka/lesson spring-boot:run
```

The Kafka settings (under `spring:`):

```
kafka:
  # bootstrap-servers defaults to localhost:9092 – the HOST listener of compose.yaml
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    acks: all # the broker confirms only after the write
    ↪ is replicated
  consumer:
    auto-offset-reset: earliest # a new group starts at the beginning of
    ↪ the topic
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    # ErrorHandlingDeserializer: a message that is not valid JSON goes to the error
    ↪ handler (→ DLT)
    # instead of failing the consumer forever
    value-deserializer:
    ↪ org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  properties:
```

```

    spring.deserializer.value.delegate.class:
↪ org.springframework.kafka.support.serializer.JsonDeserializer
    spring.json.trusted.packages: com.springbootedu.messagingkafka.*
streams:
    application-id: bookstore-sales-counter    # also the consumer group of the Streams
↪ application

```

i Note

Spring Boot 4.1 has no Docker Compose service connection for Kafka. None is needed here: `spring.kafka.bootstrap-servers` defaults to `localhost:9092`, the listener that `compose.yaml` publishes for applications on your machine.

3.1 Producing Events

The event is a record, serialized as JSON by `JsonSerializer`:

```

@Component
public class OrderEvents {

    public static final String TOPIC = "orders";
    public static final String DEAD_LETTER_TOPIC = "orders.DLT";

    private final KafkaTemplate<String, OrderPlaced> kafka;

    public OrderEvents(KafkaTemplate<String, OrderPlaced> kafka) {
        this.kafka = kafka;
    }

    public CompletableFuture<SendResult<String, OrderPlaced>> publish(OrderPlaced order)
↪ {
        return kafka.send(TOPIC, order.orderId(), order);           // asynchronous:
↪     returns at once
    }
}

```

- `send` is asynchronous. The returned `CompletableFuture` completes when the broker has confirmed the write. With `acks: all`, that is after all in-sync replicas have it.
- The serializer also writes a `__TypeId__` header with the Java type, which the consumer uses (see `spring.json.trusted.packages`).

Topics are declared as beans. Boot's `KafkaAdmin` creates them on startup:

```

@Configuration(proxyBeanMethods = false)
public class TopicConfiguration {

```

```

@Bean
NewTopic ordersTopic() {
    return TopicBuilder.name(OrderEvents.TOPIC).partitions(3).replicas(1).build();
    ↪ // 3 = max. parallel consumers
}

@Bean
NewTopic ordersDeadLetterTopic() {
    return TopicBuilder.name(OrderEvents.DEAD_LETTER_TOPIC).partitions(1).replicas(1).build();
}

```

```

== 3.1 Produce
sent to orders-1 at offset 0

```

3.2 Consuming Events

```

@Component
class ShippingListener {

    private static final Logger log = LoggerFactory.getLogger(ShippingListener.class);

    private final Shipments shipments;

    ShippingListener(Shipments shipments) {
        this.shipments = shipments;
    }

    @KafkaListener(topics = OrderEvents.TOPIC, groupId = "shipping")
    void onOrderPlaced(OrderPlaced order) { // JSON → record
        ↪ by the deserializer
        if (order.quantity() <= 0) {
            throw new InvalidOrderException("quantity must be positive, was " +
                ↪ order.quantity());
        }
        shipments.ship(order);
        log.info("Shipping {} × {} for order {}", order.quantity(), order.isbn(),
        ↪ order.orderId());
    }
}

```

- @KafkaListener starts a listener container that polls the topic and calls the method for every record.

- The `ErrorHandlingDeserializer` from the settings turns the JSON into an `OrderPlaced` before the method is called.
- `InvoiceListener` (section 3.4) listens to the same topic in group invoicing. Both groups receive every event.

Look at the groups and their **lag** (how far behind they are):

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --group shipping
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
shipping	orders	0	3	3	0
shipping	orders	1	3	3	0
shipping	orders	2	2	2	0

3.3 Errors, Retries and the Dead Letter Topic

When a listener throws, Spring Kafka's error handler decides what happens. Without precautions, one record that always fails would be retried forever, and the partition behind it would be stuck. Our handler retries twice and then parks the record in the **dead letter topic** `orders.DLT`:

```
@Configuration(proxyBeanMethods = false)
public class ErrorHandlingConfiguration {

    @Bean // Boot adds a CommonErrorHandler
    ↪ bean to every listener
    DefaultErrorHandler kafkaErrorHandler(ProducerFactory<Object, Object> producers) {
        var recoverer = new DeadLetterPublishingRecoverer(deadLetterTemplate(producers),
            (record, exception) -> new TopicPartition(OrderEvents.DEAD_LETTER_TOPIC,
                ↪ -1)); // -1: any partition
        var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(200, 2)); // 1
            ↪ try + 2 retries, 200 ms apart
        handler.addNotRetryableExceptions(InvalidOrderException.class); //
    ↪ hopeless: straight to the DLT
        return handler;
    }
}
```

- `InvalidOrderException` is not retryable: an order with quantity 0 will never become valid, so it goes to the DLT at once.
- The recoverer adds headers such as `kafka_dlt-exception-message` and `kafka_dlt-original-topic`, so the record can be analysed and replayed later.
- A **poison pill** is a record that cannot even be deserialized (e.g. this is not json). The `ErrorHandlingDeserializer` catches the error and hands the record to the error

handler, which republishes the **original bytes** to the DLT. Without it, the consumer would fail on the same record in an endless loop.

DeadLetterTest checks both cases.

3.4 Non-Blocking Retries with @RetryableTopic

Retrying in place blocks the partition while waiting. If a downstream service is down for a while, it is better to move the failed record out of the way and retry it later:

```
@Component
class InvoiceListener {

    private static final Logger log = LoggerFactory.getLogger(InvoiceListener.class);

    private final Invoices invoices;

    InvoiceListener(Invoices invoices) {
        this.invoices = invoices;
    }

    @RetryableTopic(attempts = "3", // 1 try + 2 retries
        backOff = @BackOff(delay = 500, multiplier = 2), // 500 ms, then 1 s
        retryTopicSuffix = "-invoicing-retry", //
        ↪ orders-invoicing-retry-500, -1000
        dltTopicSuffix = "-invoicing-dlt", //
        ↪ orders-invoicing-dlt
        sameIntervalTopicReuseStrategy =
        ↪ SameIntervalTopicReuseStrategy.MULTIPLE_TOPICS)
    @KafkaListener(topics = OrderEvents.TOPIC, groupId = "invoicing")
    void onOrderPlaced(OrderPlaced order) {
        int attempt = invoices.countAttempt(order.orderId());
        if (order.customerId().startsWith("flaky-") && attempt < 3) {
            throw new IllegalStateException("invoicing service unavailable (attempt " +
                ↪ attempt + ")");
        }
        invoices.invoice(order.orderId());
        log.info("Invoiced order {} after {} attempt(s)", order.orderId(), attempt);
    }

    @DltHandler
    void onGiveUp(OrderPlaced order) {
        log.warn("Invoicing gave up on order {}", order.orderId());
    }
}
```

Spring Kafka creates the retry topics and the DLT itself and runs a listener on each:

```
orders —fail→ orders-invoicing-retry-500 —fail→ orders-invoicing-retry-1000
↪ —fail→ orders-invoicing-dlt
```

The topic names end with the delay in milliseconds. The main topic keeps flowing while a failed record waits. The price is that events of the same order can now be processed **out of order**.

```
== 3.2 Consume (group shipping, group invoicing)
flaky order invoiced after 3 attempts
```

3.5 Kafka Transactions

Moving stock between warehouses produces two messages: a withdrawal and a deposit. A consumer must never see only one of them. A Kafka transaction makes several writes atomic:

```
@Component
public class StockTransfers {

    public static final String TOPIC = "stock-transfers";

    private final DefaultKafkaProducerFactory<String, String> transactionalProducers;
    private final KafkaTemplate<String, String> kafka;

    public StockTransfers(ProducerFactory<Object, Object> producers) {
        this.transactionalProducers = new
            ↪ DefaultKafkaProducerFactory<>(producers.getConfiguratioProperties(),
                new StringSerializer(), new StringSerializer());
        transactionalProducers.setTransactionIdPrefix("stock-transfer-"); // makes the
        ↪ producers transactional
        this.kafka = new KafkaTemplate<>(transactionalProducers);
    }

    public void transfer(String transferId, String from, String to, int quantity, boolean
        ↪ failInTheMiddle) {
        kafka.executeInTransaction(operations -> {
            operations.send(TOPIC, transferId, "withdraw " + quantity + " from " + from);
            operations.flush(); // really written to
            ↪ the log ...
            if (failInTheMiddle) {
                throw new IllegalStateException("failure after the first message"); // ...
                ↪ and then aborted
            }
            operations.send(TOPIC, transferId, "deposit " + quantity + " to " + to);
        });
    }
}
```

```
        return null;
    });
}
```

- `setTransactionIdPrefix` makes the producer transactional. `executeInTransaction` commits when the callback returns and aborts when it throws.
- An aborted message may already be in the log. Consumers with `isolation.level=read_committed` skip it. `read_uncommitted` consumers (the default of the Kafka client) still see it. `KafkaTransactionTest` shows both.

! Important

A Kafka transaction covers **only Kafka**. It cannot include a PostgreSQL write. For “save to the database and send an event” you need the outbox (next section).

3.6 The Transactional Outbox

“Save the order, then send the event” has two failure windows. If the application crashes after the commit but before the send, the event is lost. If it sends first and the commit then fails, the event describes an order that does not exist. The **outbox pattern** puts the event into the same database transaction:

```
CREATE TABLE outbox (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,    -- also the sending
    ⇨ order
    aggregate_id VARCHAR(36) NOT NULL,                                -- becomes the Kafka key
    event_type   VARCHAR(100) NOT NULL,
    payload      TEXT        NOT NULL,                                -- the event as JSON
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    sent_at      TIMESTAMPTZ                                     -- NULL = not yet sent
);

CREATE INDEX outbox_unsent ON outbox (id) WHERE sent_at IS NULL;    -- the relay only
⇨ reads unsent rows
```

```
@Service
public class OrderService {

    private final JdbcClient jdbc;
    private final JsonMapper json;

    public OrderService(JdbcClient jdbc, JsonMapper json) {
        this.jdbc = jdbc;
        this.json = json;
    }
}
```

```

@Transactional
public OrderPlaced place(String customerId, String isbn, int quantity) {
    OrderPlaced event = new OrderPlaced(UUID.randomUUID().toString(), customerId,
    ↪ isbn, quantity);

    jdbc.sql("INSERT INTO orders (id, customer_id, isbn, quantity) VALUES (:id,
    ↪ :customer, :isbn, :quantity)")
        .param("id", event.orderId()).param("customer", customerId)
        .param("isbn", isbn).param("quantity", quantity)
        .update();
    jdbc.sql("INSERT INTO outbox (aggregate_id, event_type, payload) VALUES (:id,
    ↪ :type, :payload)")
        .param("id", event.orderId())
        .param("type", OrderPlaced.class.getSimpleName())
        .param("payload", json.writeValueAsString(event))
        .update();
    reserveStock(isbn, quantity); // may fail → the order AND the
    ↪ event are rolled back
    return event;
}

```

If the stock check fails, the order **and** its outbox row are rolled back together. A relay sends the rows later:

```

@Component
public class OutboxRelay {

    record PendingEvent(long id, String payload) {
    }

    private final JdbcClient jdbc;
    private final JsonMapper json;
    private final OrderEvents events;

    public OutboxRelay(JdbcClient jdbc, JsonMapper json, OrderEvents events) {
        this.jdbc = jdbc;
        this.json = json;
        this.events = events;
    }

    @Scheduled(fixedDelayString = "${bookstore.outbox.relay-interval}")
    @Transactional
    public int relay() throws ExecutionException, InterruptedException, TimeoutException
    ↪ {
        List<PendingEvent> pending = jdbc.sql("""
            SELECT id, payload FROM outbox

```

```

        WHERE sent_at IS NULL
        ORDER BY id
        LIMIT 100
        FOR UPDATE SKIP LOCKED""") // several relays
↪ never send the same row
        .query(PendingEvent.class)
        .list();
        for (PendingEvent event : pending) {
            events.publish(json.readValue(event.payload(), OrderPlaced.class))
                .get(10, TimeUnit.SECONDS); // wait for the
            ↪ broker's acknowledgement
            jdbc.sql("UPDATE outbox SET sent_at = now() WHERE id = :id").param("id",
↪ event.id()).update();
        }
        return pending.size();
    }
}

```

- FOR UPDATE SKIP LOCKED lets several application instances run the relay at the same time without sending a row twice.
- The relay waits for the broker's acknowledgement before it marks a row as sent. If it crashes in between, the row is sent again after the restart: at-least-once, so consumers must be idempotent.
- Tools such as Debezium read the outbox table from the database log instead of polling. The pattern stays the same.

3.7 Kafka Streams

Kafka Streams is a library for processing topics continuously inside your application. This topology keeps the number of sold copies per book:

```

@Configuration(proxyBeanMethods = false)
@EnableKafkaStreams
public class SalesCounter {

    public static final String STORE = "sales-per-book";
    public static final String OUTPUT_TOPIC = "sales-per-book";

    @Bean
    public KStream<String, OrderPlaced> salesPerBook(StreamsBuilder builder) {
        var orderSerde = new JacksonJsonSerde<>(OrderPlaced.class).ignoreTypeHeaders();

        KStream<String, OrderPlaced> orders =
            builder.stream(OrderEvents.TOPIC, Consumed.with(Serdes.String(),
↪ orderSerde));
    }
}

```

```

        orders.filter((orderId, order) -> order != null && order.quantity() > 0)
            .groupBy((orderId, order) -> order.isbn(), Grouped.with(Serdes.String(),
                ↪ orderSerde)) // re-key
            .aggregate(() -> 0L, (isbn, order, total) -> total + order.quantity(),
                Materialized.<String, Long, KeyValueStore<Bytes, byte[]>>as(STORE)
                ↪ // local state
                    .withKeySerde(Serdes.String()).withValueSerde(Serdes.Long()))
            .toStream()
            .to(OUTPUT_TOPIC, Produced.with(Serdes.String(), Serdes.Long()));
            ↪ // every update
        return orders;
    }
}

```

- `groupBy` changes the key from order id to ISBN. Kafka Streams writes the records through an internal repartition topic for this.
- `aggregate` keeps the running totals in a local **state store**, backed by a changelog topic. After a restart, the totals are restored from it, so the count continues where it stopped.
- `@EnableKafkaStreams` plus `spring.kafka.streams.application-id` start the topology with the application.

`SalesCounterTopologyTest` tests the topology with `TopologyTestDriver`: no broker, no Docker, milliseconds per test.

! Warning

The local state lives in `spring.kafka.streams.state-dir` (by default under the temp directory). If you point the application at a new, empty Kafka, delete this directory: the old local state does not match the new broker.

3.8 Testing with Testcontainers

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final KafkaContainer KAFKA = new KafkaContainer("apache/kafka:4.2.1");
    ↪ // KRaft, no ZooKeeper

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
    ↪ pg18").asCompatibleSubstituteFor("postgres"));

    @Bean
    @ServiceConnection // sets
    ↪ spring.kafka.bootstrap-servers

```

```
KafkaContainer kafkaContainer() {  
    return KAFKA;  
}  
  
@Bean  
@ServiceConnection  
PostgreSQLContainer postgresContainer() {  
    return POSTGRES;  
}  
}
```

- `org.testcontainers.kafka.KafkaContainer` runs the official `apache/kafka` image in KRaft mode. `@ServiceConnection` sets the bootstrap servers.
- Messaging is asynchronous: tests send an event and then wait with `Awaitility` (`await().atMost(...)`) until the effect is visible. Never use a fixed `Thread.sleep`.
- Every test uses new random order ids, so tests do not see each other's events on the shared topics.

4. Common Mistakes and Best Practices

!! Caution

Never send a Kafka message inside a database transaction and assume both happen together. Use the outbox pattern, or accept and handle the gap explicitly.

- **Do:** choose the key by the ordering you need. Events of one order must share a key.
- **Don't:** create more consumers in a group than the topic has partitions. The extra consumers stay idle.
- **Do:** make consumers idempotent. At-least-once delivery means duplicates will happen.
- **Don't:** let one bad record block a partition. Configure an error handler with a dead letter topic, and use an `ErrorHandlingDeserializer`.
- **Do:** set `spring.json.trusted.packages` to your own packages only. Deserializing arbitrary types from headers is a security risk.
- **Don't:** expect ordering across partitions, or after non-blocking retries.

5. Summary

- Kafka stores events in partitioned, ordered logs. The key decides the partition and thereby the order.
- Each consumer group receives every event once. Within a group, the partitions are split among the instances.

- `DefaultErrorHandler` retries, then sends hopeless records and poison pills to a dead letter topic.
- `@RetryableTopic` retries through separate topics, without blocking the main topic.
- Kafka transactions make several writes atomic for `read_committed` consumers, but only within Kafka.
- The transactional outbox saves the event together with the data, and a relay publishes it at least once.
- Kafka Streams processes topics continuously with local, fault-tolerant state. `TopologyTestDriver` tests it without a broker.

Next step: [Exercises](#)

6. Further Reading

- [Spring for Apache Kafka Reference](#)
- [Spring Kafka — Handling Exceptions · Non-Blocking Retries · Transactions](#)
- [Spring Kafka — Kafka Streams Support](#)
- [Spring Boot — Apache Kafka Support](#)
- [Apache Kafka Documentation](#)
- [Pattern: Transactional Outbox](#)